Software Architecture and Design Specification

Project: Canteen Pre-Order System

Authors: Student Team

Version: 1.0

Date: 11-10-2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	11-10-2026	Student Team	

Approvals

Role	Name	Signature/Date
Course Coordinator	Arpitha K	

1. Introduction

1.1 Purpose

This Software Architecture and Design Specification (SAD) defines the architecture, major components, interfaces, data responsibilities, security design, and detailed design of the Canteen Pre-Order System. It translates the requirements in the SRS into an implementable software design.

1.2 Scope

The system covers student registration and authentication, menu browsing and filtering, cart management, pre-order placement, payment-status handling, order tracking and history, eligible cancellation, staff authentication, menu management, order processing, status updates, and basic operational reporting. Physical food preparation and the internal implementation of external payment providers remain outside the system scope.

1.3 Audience

Students, instructors, developers, QA engineers, system integrators, canteen staff, maintenance teams, and assessment evaluators.

1.4 Definitions

SRS – Software Requirements Specification; SAD – Software Architecture and Design Specification; UI – User Interface; API – Application Programming Interface; DB – Database; RBAC – Role-Based Access Control; RTM – Requirements Traceability Matrix; UAT – User Acceptance Testing.

2. Document Overview

2.1 How to use this document

This document describes the system architecture and design corresponding to the SRS. It provides the component architecture, technology stack, architecture rationale, security architecture, UML sequence flows, API design, error handling, UX considerations, risks, and traceability to requirements.

2.2 Related Documents

Primary related document: Canteen Pre-Order System SRS, Version 1.0, dated 01-10-2026. The SRS defines the functional requirements, non-functional requirements, security objectives and requirements, acceptance tests, UML use cases, and RTM.

3. Architecture

3.1 Goals & Constraints

Architecture goals:

- Separate frontend, backend, authentication, business logic, and persistence responsibilities.
- Provide secure authenticated access for students and authorized canteen staff.
- Maintain consistent order data, server-side totals, and valid order-status transitions.
- Provide responsive and understandable web workflows for students and staff.
- Support growth in users, menu items, and orders without redesigning core modules.

Constraints:

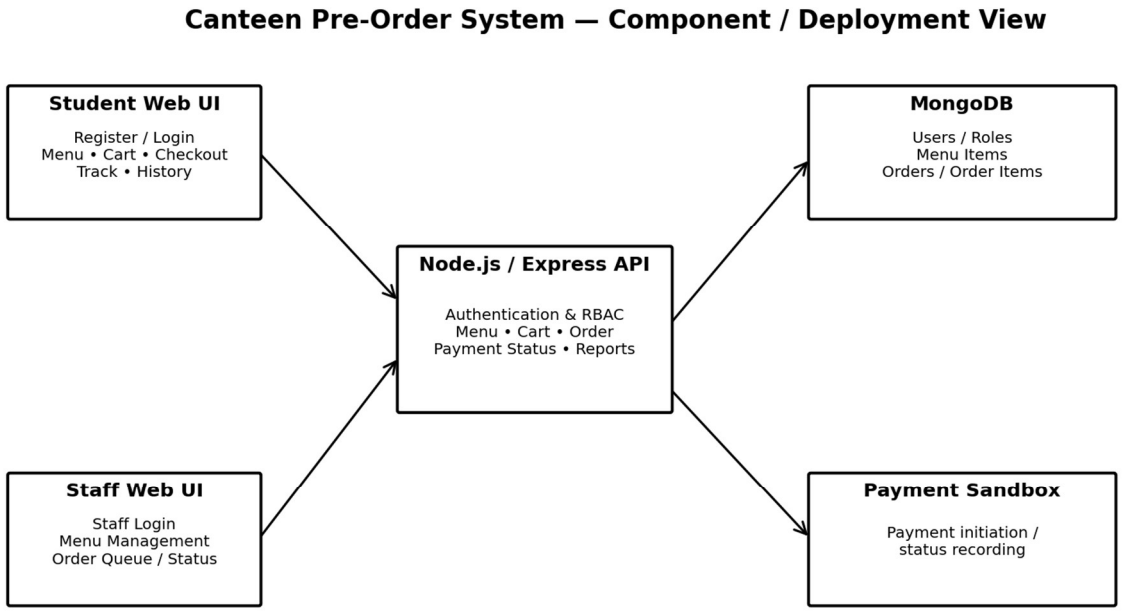
- The system is a web-based client-server application and depends on network and database availability.
- Only authorized staff may perform staff operations.
- The academic implementation may use a simulated or sandbox payment mechanism.
- The SRS suggests React/JavaScript, Node.js/Express, and MongoDB as a suitable implementation stack.
- No specialized hardware is required for the core system.

3.2 Stakeholders & Concerns

Stakeholder	**Primary Concerns**
Student	Simple navigation, accurate menu/prices, secure login, cart and clear order status.
Canteen Staff	Fast order queue processing, menu/availability management, valid status transitions.
System Administrator	Authorized configuration and maintenance.
Developers	Modularity, separation of responsibilities, maintainability.
QA Engineers	Testability, acceptance criteria, traceability to requirements.
System Integrators	Stable APIs, JSON communication, payment-status integration.
Assessment Evaluators	Requirement coverage, architecture rationale, UML/design evidence.

3.3 Component (UML) Diagram

The architecture follows a client-server structure with separate student/staff web interfaces, an Express API layer, persistent MongoDB storage, and an external/sandbox payment interface.



3.4 Component Descriptions

Component	Responsibilities
Student Web UI	Registration/login screens, menu browsing/search/filtering, cart, checkout, order confirmation, tracking, history and cancellation.
Staff Web UI	Staff login, menu CRUD and availability, incoming-order queue, order acceptance/rejection, preparation/ready/completed updates, reports.
Authentication & RBAC	Validates credentials, establishes authenticated sessions/tokens, and restricts staff functions to authorized accounts.
Menu Module	Retrieves menu data and supports authorized staff CRUD and availability changes.
Cart & Order Module	Validates item availability/quantity, calculates server-side totals, creates unique orders and controls eligible cancellation.
Payment Status Module	Initiates payment or records payment status; may use a sandbox/simulation for academic implementation.
Tracking & Reporting Module	Provides current order status, order history and basic period-based order/sales summaries.
MongoDB Database	Persists users, roles, menu items, orders, order items, payment status and related timestamps.
Optional Notification Service	Provides notification capability if included in the implementation; it is not required for the core system.

3.5 Chosen Architecture Pattern and Rationale

A layered client-server architecture is selected. The presentation layer contains the responsive web interfaces; the API/application layer contains authentication, authorization and business modules; and the data layer manages persistent MongoDB storage. This separation matches the SRS requirement that frontend, backend, database, and authentication responsibilities be separated into maintainable modules. A full microservices architecture is not specified by the SRS and would add deployment and operational complexity for the academic system.

Layer	Main Responsibility
Presentation Layer	Student and staff web UI, forms, validation messages, navigation and status display.
API/Application Layer	Authentication, RBAC, menu, cart, order, payment-status, tracking and reporting logic.
Data Layer	MongoDB persistence, retrieval, order consistency and timestamped records.
External Integration	Payment sandbox/status interface and optional notification service.

3.6 Technology Stack & Data Stores

Area	Technology / Design
Frontend	React with JavaScript; responsive web interface.
Backend	Node.js with Express; HTTP/HTTPS JSON APIs.
Database	MongoDB for persistent application data.
Communication	HTTP/HTTPS; HTTPS required in deployed environments.
Data Format	JSON request/response format.
Authentication	Authenticated session/token mechanism with protected API endpoints.
Authorization	Role-Based Access Control (RBAC) for staff operations.
Payment	Sandbox/simulated payment mechanism or supported payment-status integration.

3.7 Risks & Mitigations

Risk	Mitigation / Design Response
Network interruption during order placement	Use request validation and idempotent/order-creation handling so retries do not create duplicate orders.
Unauthorized staff operation	Authenticate staff and enforce RBAC on staff-only endpoints.
Client-side price manipulation	Calculate totals on the server using stored item prices and submitted quantities.
Ordering unavailable items	Re-check item availability on the server before creating the order.
Invalid order-state transition	Validate allowed transitions such as Accepted → Preparing → Ready → Completed.
Sensitive information exposure in logs	Do not log passwords, credentials, or authentication tokens; use standardized errors.
Database inconsistency	Persist confirmed order items, quantities, prices and totals consistently and test data integrity.

3.8 Traceability to Requirements

SRS Requirement	Architecture / Design Element
CPS-F-001, CPS-F-002	Authentication module and User data store
CPS-F-003, CPS-F-004	Menu module and Student Web UI

CPS-F-005, CPS-F-006	Cart & Order module with server-side validation
CPS-F-007, CPS-F-008	Order creation and unique order identifier
CPS-F-009	Payment Status module / sandbox integration
CPS-F-010, CPS-F-011, CPS-F-012	Tracking, history and cancellation logic
CPS-F-013, CPS-F-014	Staff authentication, RBAC and Menu Admin functions
CPS-F-015, CPS-F-016	Staff order queue and order-state management
CPS-F-017, CPS-F-018	Server-side availability and total validation
CPS-F-019, CPS-F-020	Validation/error handling and reporting module
CPS-NF-001 to CPS-NF-008	Layer separation, performance, reliability, accessibility, audit logging, scalability, compatibility and data integrity design
CPS-SR-001 to CPS-SR-006	Authentication, RBAC, server-side validation, secure logging and HTTPS

3.9 Security Architecture

The security architecture implements the SRS security objectives: protect user accounts and order information from unauthorized access; ensure only authorized staff can perform menu and order-management operations; and maintain the integrity and confidentiality of order, payment-status, and user data.

Threat / Security Concern	**Design Control**
Credential compromise	One-way password hashing; passwords are never stored in plaintext.
Unauthorized API access	Protected endpoints verify authentication before protected operations.
Unauthorized staff access	Staff-only endpoints enforce role-based authorization.
Price/order manipulation	Server validates prices, quantities, item IDs and order totals.
Sensitive information leakage	Credentials and authentication tokens are excluded from application logs.
Network interception	HTTPS is required in deployed environments for protected communication.

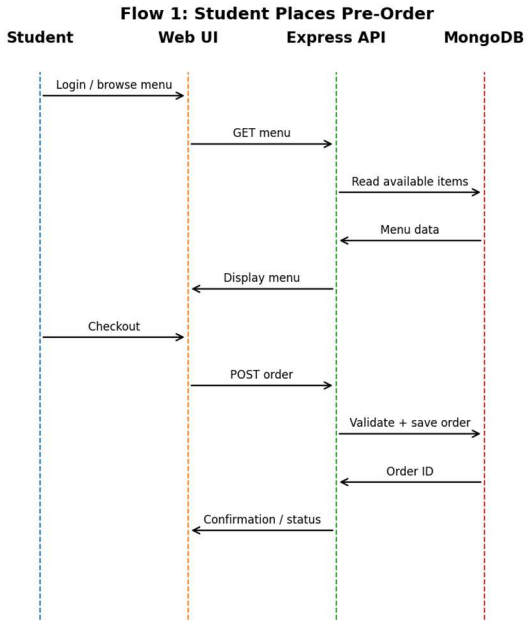
4. Design

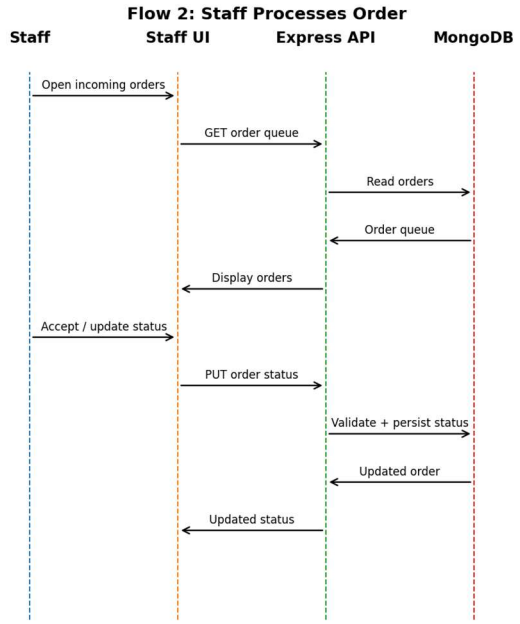
4.1 Design Overview

The design uses modular frontend and backend responsibilities. A student or staff browser sends JSON requests to the Express API. The API authenticates the request, applies role and business-rule validation, performs the required operation, and reads/writes persistent data in MongoDB. Payment-status operations can connect to a sandbox/simulated payment mechanism. The design keeps server-side validation authoritative for availability, quantities, prices, totals, permissions and order-state transitions.

4.2 UML Sequence Diagrams

The following two sequence flows cover the principal student ordering flow and the canteen staff order-processing flow.





Sequence Flow 1 — Student places a pre-order: the UI obtains menu data through the API, the API reads MongoDB, and checkout results in a validated order being persisted and confirmed.

Sequence Flow 2 — Staff processes an order: the staff UI retrieves the incoming order queue, the staff submits a valid status change, and the API persists the updated status in MongoDB.

4.3 API Design

The SRS specifies authentication, menu, order, payment-status and database interfaces, using HTTP/HTTPS and JSON. The following representative interfaces translate those requirements into the SAD design.

Component / Endpoint	Method	Request	Response / Errors
Authentication API — /api/auth/register	POST	name, email/username, password and required registration details	Account creation result; validation/duplicate registration error.
Authentication API — /api/auth/login	POST	registered credentials	Authenticated session/token; invalid credentials rejected.
Menu API — /api/menu	GET	Optional search/filter parameters	Available menu items with name, category, description, price and availability.
Order API — /api/orders	POST	Selected item IDs, quantities and required order details	Unique order ID, calculated total and confirmation; unavailable/invalid quantities rejected.
Order Tracking API — /api/orders/{id}	GET	Authenticated order identifier	Current order status and order details; unauthorized access rejected.
Staff Order API — /api/staff/orders/{id}/status	PUT	Requested valid status	Updated order status; invalid state transition or

			unauthorized access rejected.

4.4 Error Handling, Logging & Monitoring

- Use clear validation messages for invalid input and unsuccessful operations without exposing internal implementation details.
- Validate important business rules on the server even when client-side validation is present.
- Reject invalid credentials, unauthorized staff operations, unavailable items, invalid quantities and invalid order-state transitions.
- Calculate order totals from stored prices and quantities on the server.
- Maintain timestamped application and order-processing logs sufficient for troubleshooting, security review and operational auditing.
- Never write sensitive credentials or authentication tokens to application logs.
- Monitor API response performance, order-processing failures, application availability and relevant security events.

4.5 UX Design

The primary interface is a responsive web application with clear navigation for menu, cart, checkout, order tracking, and staff order/menu management. Forms should provide validation messages and status information. The design supports keyboard navigation, readable text, labels, suitable contrast, and understandable status information for the supported core workflows.

User	Key UX Flow
Student	Register/Login → Browse/Search Menu → Add to Cart → Validate Cart → Checkout → Payment Status → Confirmation → Track/History
Canteen Staff	Staff Login → View Menu/Availability → View Incoming Orders → Accept/Reject → Preparing → Ready → Completed → Reports

4.6 Open Issues & Next Steps

- Finalize the exact authentication/session or token implementation and deployment configuration.
- Define the concrete payment sandbox/provider interface used by the academic implementation.
- Finalize database collection/schema details and indexes during implementation.
- Implement and execute the SRS acceptance test suites, including Authentication, Menu, Cart, Order Placement, Payment Status, Tracking, Staff Operations, Performance, Security, Compatibility and Usability.
- Complete the RTM status updates from N (Not started) to P/A as implementation and verification progress.

5. Appendices

5.1 Glossary

Term	Meaning
SRS	Software Requirements Specification
SAD	Software Architecture and Design Specification
API	Application Programming Interface
RBAC	Role-Based Access Control
UI	User Interface
DB	Database
RTM	Requirements Traceability Matrix

UAT	User Acceptance Testing
HTTPS	HTTP Secure communication protocol

5.2 References

1. Canteen Pre-Order System SRS, Version 1.0, 01-10-2026 — primary source for this SAD. 2. SAD Template provided for the assignment — structure and section organization.

5.3 Tools

Suggested/appropriate design and development tools from the project context: React/JavaScript, Node.js/Express, MongoDB, PlantUML or draw.io for UML diagrams, and an API testing tool such as Postman.